

Clasificación de kilonovas en Roman

Notas de estudio: datos, fotometría, grilla LANL y transformer

Proyecto Kilonova — repositorio Kilonova

1 de agosto de 2026

Resumen

Apuntes autocontenidos de todo lo construido hasta ahora: de dónde salen los datos (OpenUniverse/SNANA y la grilla LANL), con qué ecuaciones se les pone error fotométrico, cómo se recortan las ventanas tempranas, cómo se tokenizan y qué arquitectura de transformer las clasifica. Cada sección cierra con las magnitudes numéricas concretas que usa el pipeline, para poder reproducirlas de memoria.

Índice

1. Panorama	2
1.1. Objetivo	2
1.2. Cadena de datos	2
1.3. Qué NO se usa	3
2. El observatorio Roman	3
2.1. La idea en una frase	3
2.2. Qué números del observatorio entran en nuestras ecuaciones	4
2.3. Filtros de la WFI	4
3. Roman HLTDS: filtros, tiers y cadencia	5
3.1. Bandas y tiers	5
3.2. Cadencia	5
4. OpenUniverse (SNANA): los contaminantes	5
4.1. Qué es y de dónde sale	5
4.2. ¿Qué es exactamente el <i>dimming</i> cosmológico?	6
4.3. Códigos <i>gentype</i>	7
4.4. Censo de la clase <i>other</i>	7
5. Receta de error fotométrico	8
5.1. La ecuación CCD	8
5.2. Flujo de la fuente y magnitud límite	8
5.3. Ruido de lectura <i>up-the-ramp</i>	8
5.4. Fondo: cielo, térmico, dark	9
5.5. NEA tabulada, no calculada	9
5.6. Jitter del punto cero	9
5.7. La misma ecuación en el lenguaje de SNANA (Hourglass)	9
5.8. Referencias de la receta	10

6. La grilla LANL de kilonovas	10
6.1. Parámetros	10
6.2. Ejes continuos: tiempo y longitud de onda	11
6.3. El bug grande: el factor angular 54	11
6.4. Del espectro en reposo a la magnitud AB observada	12
6.5. Muestreo de ventanas KN	12
6.6. Validación contra Chase et al. (2021)	13
7. La ventana temprana	13
8. Tokens	14
8.1. Un token = una medición (objeto, visita, banda)	14
8.2. Normalización global	14
8.3. Tokens globales y regímenes de redshift	14
8.4. Augmentaciones	15
8.5. Sin token de modo deep/wide	15
9. El transformer	15
9.1. Time2Vec	15
9.2. Fusión del token	17
9.3. Atención, paso a paso	17
9.4. Bloque encoder (pre-norm)	19
9.5. Hiperparámetros entrenados	20
9.6. Receta de entrenamiento	20
9.7. Split anti-fuga	21
10.Resultados	21
10.1. Matriz de degradación (sin redshift)	22
10.2. Recall de KN en el punto de operación <i>recall-first</i>	22
11.Apéndice: preguntas típicas	22
Fuentes dentro del repositorio	25

1. Panorama

1.1. Objetivo

Clasificar **kilonovas (KN)** frente a **contaminantes** (SN, TDE, SLSN, PISN) en las curvas de luz del *High Latitude Time Domain Survey* (HLTDS) del Roman Space Telescope, usando solo las **primeras épocas** de la curva. El criterio operativo es **recall sobre precisión**: la meta es aislar candidatos para seguimiento espectroscópico, y el umbral se calibra contra un presupuesto de seguimiento, no contra la prevalencia real.

1.2. Cadena de datos

```

LANL .dat (900 sims x 54 angulos)
--kn-cache-lanl--> lanl_spectra.parquet    (espectros rest-frame, 1024 lambda)
--kn-kilonova-windows--> kn_windows_{deep,wide}.parquet    [clase KN]

OpenUniverse snana (hdf5 SED + mags verdaderas, parquet cabecera)
--kn-run-openuniverse--> early_windows_{deep,wide}.parquet    [clase other]

ambos --> openuniverse_data.py (tokens) --> KilonovaTransformer --> {other, KN}

```

Las dos ramas comparten **exactamente la misma** receta de ruido, cadencia y lógica de ventana (`build_window_from_model` en `src/kilonova/simulation/early_windows.py`). Eso es deliberado: si las recetas difirieran, el clasificador aprendería “mi pipeline vs. el suyo” en vez de “KN vs. no KN” (*injection bias*).

1.3. Qué NO se usa

- **Hourglass** (Rose et al. 2025) ya no genera datos de entrenamiento; sobrevive solo como *fuelle de la receta de ruido* y como validación de la misma.
- El plan original de dos etapas (Hourglass → OpenUniverse) está superado: se entrena *solo* con OpenUniverse.
- La construcción de *matched twins* (una KN gemela por cada contaminante de bajo z) se descartó: las KN de LANL son demasiado débiles para tener análogas entre los contaminantes de OpenUniverse. Las KN son ahora una población independiente sobre una grilla de z .
- `simulation/extinction.py` y el CLI `kn-extinguish`: es una rama paralela que *no* genera los datos de entrenamiento (sección 6.4). Ni tampoco `src/kilonova/datasets/openuniverse.py`, un segundo tokenizador divergente — el que entrenó el modelo es `training/openuniverse_data.py`.
- La cadencia y las exposiciones propias de OpenUniverse: del catálogo solo se toman las curvas de luz verdaderas, y encima se monta la cadencia HLTDS de este proyecto.

2. El observatorio Roman

2.1. La idea en una frase

El *Nancy Grace Roman Space Telescope* es, en resolución y sensibilidad, **un Hubble**; la diferencia es el **campo de visión**, unas **200 veces** el del infrarrojo de Hubble. Mismo espejo primario de **2.4 m** (de hecho es un telescopio AFTA donado a la NASA), misma calidad de imagen, pero un plano focal enorme detrás. Eso convierte la astronomía de “apuntar a un objeto” en astronomía de *survey*: es lo que hace posible que exista una muestra estadística de transientes como la que usamos aquí.

Espejo primario	2.4 m de diámetro (el mismo que Hubble)
Apertura efectiva	2.36 m de diámetro con obstrucción central 0.32; $A = 37\,570\text{ cm}^2 = 3.757\text{ m}^2$ (el anillo puro daría 3.93 m^2 : galsim descuenta además los <i>struts</i>)
Campo de visión (WFI)	caja $0.8^\circ \times 0.4^\circ = 0.32\text{ deg}^2$; área activa 0.281 deg^2 con los huecos entre detectores ya descontados
Plano focal	18 detectores Teledyne H4RG-10 de $4096 \times 4096\text{ px}$ (4088 activos) $\Rightarrow \sim 300\text{ megapíxeles}$
Escala de placa	0.11 arcsec/píxel (píxel físico de $10\text{ }\mu\text{m}$)
PSF	FWHM $0.058''$ en F062 a $0.169''$ en F213
Rango espectral	$0.48\text{--}2.3\text{ }\mu\text{m}$ (WFI); coronógrafo aparte en $0.4\text{--}1.0\text{ }\mu\text{m}$
Temperatura	detectores a 89.5 K, enfriamiento pasivo por radiadores
Órbita	punto de Lagrange L2 Sol-Tierra
Misión	5 años nominales (objetivo 10); lanzamiento 30 de agosto de 2026

La comparación con Hubble, hecha a mano. WFC3/IR tiene un campo de $123'' \times 136'' = 4.65\text{ arcmin}^2 = 1.29 \times 10^{-3}\text{ deg}^2$. Entonces

$$\frac{0.281}{1.29 \times 10^{-3}} \approx 218 \quad (1)$$

Ojo con la cifra que se cita. La NASA dice públicamente “*at least 100 times larger than Hubble’s*”, sin nombrar instrumento; no dice 200. Los dos números son compatibles y la diferencia es contra qué cámara de Hubble se compare:

referencia	FOV	Roman/HST
WFC3/IR ($123'' \times 136''$)	4.65 arcmin ²	218
ACS/WFC ($202'' \times 202''$, la cámara de campo más ancho de HST)	11.33 arcmin ²	89

El “al menos 100” de la NASA es un piso conservador medido contra ACS/WFC; el 200 es la comparación en el infrarrojo, que es la relevante aquí. **Citar cuál de las dos se usa.** La sensibilidad por exposición es parecida porque el área colectora es la misma; lo que cambia es *cuánto cielo* se cubre por unidad de tiempo.

2.2. Qué números del observatorio entran en nuestras ecuaciones

No todo el observatorio importa para este trabajo. Los que aparecen literalmente en el código (vía `galsim.roman`) son:

constante	dónde entra	valor
<code>collecting_area</code>	flujo de la fuente, $F = t_{\text{exp}} A 10^{-0.4(m-ZP)}$	37 570 cm ²
<code>pixel_scale</code>	conversión del cielo a e [−] /píxel	0.11 ″/px
<code>dark_current</code>	término B_{dark} del fondo	0.015 e [−] /s/px
<code>thermal_backgrounds</code>	término B_{thermal}	0.04 (H158), 0.17 (F184) e [−] /s/px
<code>getSkyLevel</code>	término B_{sky} (zodiacal, depende del campo)	—
<code>read_noise</code>	<i>no</i> se usa: lo sustituye la Ec. (13)	(8.5 e [−] estático)
<code>getPSF</code>	<i>no</i> se usa: NEA tabulada de Rose+2025	—

Un detalle que sí importa: Roman está *submuestreado*. La PSF tiene FWHM 0.058″ en el azul frente a un píxel de 0.11″: menos de un píxel por FWHM. Por eso el NEA — el número efectivo de píxeles que pesa el fondo en la Ec. (6) — es tan pequeño (5.6 px en R062) y tan sensible al modelo de PSF que se use; y por eso importa la decisión de tabularlo en vez de calcularlo (sección 5.5).

2.3. Filtros de la WFI

La nomenclatura oficial es F062, F087, ...; en este proyecto (y en `galsim.roman`) se usa R062, Z087, Y106, J129, H158, F184, con la letra del nombre clásico de la banda. Son el mismo filtro.

proyecto	oficial	rango [μm]	centro [μm]	ZP AB (galsim)
R062	F062	0.48–0.76	0.620	15.297
Z087	F087	0.76–0.98	0.869	14.964
Y106	F106	0.93–1.19	1.060	15.024
J129	F129	1.13–1.45	1.293	15.040
H158	F158	1.38–1.77	1.577	15.074
F184	F184	1.68–2.00	1.842	14.622
(K213)	F213	1.95–2.30	2.125	14.579

K213 existe en el instrumento pero no se usa en el HLTDS: su fondo térmico es 4.52 e[−]/s/px, un **factor 27** por encima de F184 (0.17), y lo haría inútil para fotometría profunda. Existen además un grism (G150, $R \sim 600$) y un prisma (P127, $R = 80\text{--}180$) que este proyecto no toca.

¿Por qué Roman y no Hubble/JWST para kilonovas? Porque el problema es de *estadística*, no de profundidad: las KN son raras, rápidas y rojas. Se necesita (i) infrarrojo — la KN se enrojece en días por los lantánidos —, (ii) profundidad de espacio, y (iii) mucho cielo con cadencia de días. Hubble y JWST tienen (i) y (ii) pero no (iii); los surveys terrestres tienen (iii) pero no (i)+(ii). Roman es la primera instalación con las tres.

3. Roman HLTDS: filtros, tiers y cadencia

3.1. Bandas y tiers

Seis filtros de la WFI participan en el HLTDS (K213 queda fuera). El survey tiene dos *tiers*: **wide** (más área, menos profundidad) y **deep**.

t_{exp} por época [s]	R062	Z087	Y106	J129	H158	F184
wide	60	85	95	152	294	—
deep	—	193	294	307	420	1636

Índices de banda del modelo: R=0, Z=1, Y=2, J=3, H=4, F=5. Los dos tiers se entrenan en un *único* modelo sobre este vocabulario de 6 bandas; una banda que un tier no observa simplemente nunca genera token.

Campos reales del HLTDS (importan porque `getSkyLevel` calcula la luz zodiacal a partir de la latitud eclíptica del campo): ELAIS-N1 (242.504, +54.510) hace *deep* y *wide*; EDfS_a (58.900, −49.320) solo *deep*; EDfS_b (63.600, −47.600) solo *wide*. **El pipeline no promedia sobre ellos: `field_center_for_tier` sortea uno con `FIELD_SEED=0`, y ese sorteo sale siempre EDfS_a para deep y EDfS_b para wide.** ELAIS-N1 está en la tabla pero nunca se usa: todos los fondos de estas notas son de esos dos campos.

3.2. Cadencia

Visitas cada $\Delta t_{\text{cad}} = 5$ d. En cada visita se observa la **banda ancla** (la más azul del tier: R062 en wide, Z087 en deep) más **dos** de las otras cuatro. Las cuatro no-ancha se parten en dos *pares consecutivos* en longitud de onda: las dos azules en visitas pares, las dos rojas en visitas impares. Resultado:

$$\text{wide: } \underbrace{\text{RZY}}_{\text{visita par}} \rightarrow \underbrace{\text{RJH}}_{\text{impar}} \rightarrow \text{RZY} \rightarrow \dots \quad \text{deep: } \text{ZYJ} \rightarrow \text{ZHF} \rightarrow \text{ZYJ} \rightarrow \dots$$

Cada banda no-ancha se observa una visita sí y otra no; el ancla, todas.

Trampa. Agrupar las bandas por *paridad del índice* en vez de por pares consecutivos da RZJ/RYP y ZYH/ZJF: los mismos conteos por banda, pero el par de filtros equivocado en cada visita y por tanto *líneas de base de color equivocadas*. Los conteos por banda no fijan la regla; el test del repositorio assera las secuencias literales.

4. OpenUniverse (SNANA): los contaminantes

4.1. Qué es y de dónde sale

OpenUniverse2024 es la simulación oficial de Roman+Rubin. De ella se usan **solo los catálogos SNANA de transientes**, no las imágenes (~400 TB). Se bajan del mirror HTTPS público de `s3://nasa-irsa-simulations/openuniverse2024/roman/full/roman_rubin_cats_v1.1.2_faint`, 33 healpix, dos ficheros por healpix:

- `snana_<healpix>.parquet` — cabecera por objeto: `id`, `gentype`, `z_CMB`, `peak_mjd`, `peak_mag_*`. ~4 MB c/u.
- `snana_<healpix>.hdf5` — grilla de SED y **magnitudes AB verdaderas por banda** en función del MJD. ~16 GB c/u, ~530 GB en total.

Punto clave: las magnitudes de OpenUniverse **ya traen la K-corrección, la atenuación por distancia y la extinción del host** incorporadas por SNANA. Lo que *no* traen es error fotométrico. Nuestro pipeline solo les aplica (i) la cadencia, (ii) la receta de ruido y (iii) la ventana temprana. Por eso ningún arreglo de la fotometría sintética toca a los contaminantes: esa fotometría solo existe en la rama de las KN, que parte de espectros LANL en reposo.

Dos precisiones que hay que saber decir. Primera: del `hdf5` se leen *solo* `mjd` y las magnitudes verdaderas por banda. **La cadencia y los tiempos de exposición propios de OpenUniverse no se usan en absoluto**; encima de esas curvas verdaderas el pipeline monta su propia grilla de visitas HLTDS de dos tiers. Segunda: la extinción de la Vía Láctea **no** está aplicada — el propio catálogo lo dice, `mw_extinction_applied` vale `False` y `mw_EBV`=0 en los 63 151 objetos. El cuadro real es:

	extinción de host	extinción Vía Láctea
contaminantes OpenUniverse	sí (dentro del modelo SED)	no
kilonovas LANL	no	no

La asimetría residual entre clases es el polvo del host, y es un sistemático conocido que hay que *declarar* (ver sección 6.4), no un bug que buscar.

4.2. ¿Qué es exactamente el *dimming* cosmológico?

Es el conjunto de efectos que hacen que un objeto de luminosidad intrínseca L se vea más débil cuanto más lejos está, *más* los factores extra que introduce la expansión del universo. Conviene separarlos porque se confunden con facilidad.

1. Dilución geométrica. El flujo cae con el cuadrado de la distancia porque los fotones se reparten sobre una esfera de área $4\pi d^2$:

$$F = \frac{L}{4\pi d^2} \quad (2)$$

Es el único efecto que existiría en un universo estático. En magnitudes es el **módulo de distancia**

$$\mu = m - M = 5 \log_{10} \left(\frac{d_L}{10 \text{ pc}} \right) \quad (3)$$

2. Los dos factores $(1+z)$ de la expansión. En un universo en expansión aparecen dos pérdidas adicionales de flujo, ambas de factor $(1+z)$:

- **Pérdida de energía por fotón:** cada fotón se estira, $\nu_{\text{obs}} = \nu_{\text{em}}/(1+z)$, así que llega con energía $h\nu$ menor en un factor $(1+z)$.
- **Dilatación temporal:** los fotones llegan más espaciados en el tiempo, en un factor $(1+z)$, de modo que la *tasa* de llegada baja otro $(1+z)$. (Este mismo factor es el que estira la curva de luz: una KN que decae en 3 d en reposo decae en $3(1+z)$ d para nosotros.)

Ambos se absorben en la definición de la **distancia de luminosidad**

$$d_L = (1+z) d_C(z), \quad d_C(z) = \frac{c}{H_0} \int_0^z \frac{dz'}{\sqrt{\Omega_m(1+z')^3 + \Omega_\Lambda}} \quad (4)$$

que es lo que garantiza que $F = L/(4\pi d_L^2)$ siga siendo válido. Es decir: los dos factores $(1+z)$ ya están dentro de d_L , no se aplican otra vez a mano. En el pipeline esto lo calcula `astropy.cosmology.Planck18` (y `FlatLambdaCDM(H0=67.4, Omega_m=0.315)` para reproducir Chase+21).

3. K-corrección. No es una atenuación, sino una *traducción de banda*: el filtro observa a λ_{obs} , pero el objeto emitió a $\lambda_{\text{obs}}/(1+z)$. Como el espectro no es plano, hay que integrar el espectro *ya redshiftado* contra la transmisión del filtro en vez de usar la magnitud en reposo. En AB:

$$m_{\text{AB}} = -2.5 \log_{10} \left(\frac{\int f_{\nu}^{\text{obs}}(\nu) T(\nu) d\nu/\nu}{\int 3631 \text{ Jy } T(\nu) d\nu/\nu} \right), \quad f_{\nu}^{\text{obs}}(\nu_{\text{obs}}) = \frac{(1+z) L_{\nu}(\nu_{\text{obs}}(1+z))}{4\pi d_L^2} \quad (5)$$

Para una KN esto es enorme, no una corrección de segundo orden: la SED es muy roja y evoluciona en horas, así que a $z = 0.3$ la banda J está viendo lo que en reposo era $\sim 1 \mu\text{m}$.

Confusión frecuente. “*Cosmological dimming*” se usa también para el **oscurecimiento de brillo superficial** $\propto (1+z)^{-4}$, que aplica a fuentes *extensas* (galaxias) y añade dos factores $(1+z)$ más por la contracción del ángulo sólido. Nuestros transientes son **puntuales**: solo aplican los dos $(1+z)$ de d_L , no cuatro.

En OpenUniverse los tres efectos ya vienen aplicados (SNANA los mete al generar `sim_mag_obs`); en la rama LANL los aplicamos nosotros explícitamente, en el orden de la sección 6.4.

4.3. Códigos `gentype`

10 SN Ia	12 SN Iax	21 SN Ib	26 SN Ic	32 SN II	40 SLSN-I
42 TDE	50 KN	57 PISN-H	58 PISN-He	99 FIXMAG	

En los `parquet` la etiqueta autoritativa es la columna `label` (string), no el `gentype`: una sola fuente de verdad, sin diccionario que mantener sincronizado río abajo. Pero **en el entrenamiento la clase no la da ninguna de las dos, sino el archivo de procedencia** (`label_by_index = where(is_kn, KN, other)`). Es seguro solo por un accidente afortunado: OpenUniverse *sí* tiene kilonovas propias (`gentype= 50`, modelos `NON1ASED.KN-K17` de Kasen+17), pero son ~ 2 por `healpix` con `peak_mag_F` ≈ 30 , así que nunca se detectan y jamás entran en `early_windows_*.parquet`. Ésa es también la respuesta a “¿por qué la grilla LANL si OpenUniverse ya trae KN?”.

`gentype= 99` (FIXMAG) se descarta al leer el catálogo, antes de la ventana.

4.4. Censo de la clase *other*

Cuidado con qué se está contando: la columna que importa para el entrenamiento es el número de **instancias por tier**, no de transientes distintos. El fichero *deep* es un superset del *wide*, y un mismo transiente observado en los dos aparece *dos veces* y el dataloader lo trata como dos objetos.

clase	n (deep+wide)	%	transientes únicos
SN II	677 050	42.4	390 141
SN Ia	443 315	27.7	223 578
SN Ic	213 400	13.4	116 881
SN Ib	190 882	11.9	105 985
SN Iax	63 250	4.0	38 237
TDE	7 238	0.5	3 720
SLSN-I	2 237	0.1	1 120
PISN-He	182	~ 0	96
PISN-H	141	~ 0	74
total other	1 597 695	100	879 832

Comprobación de que la columna buena es la primera: $1\,597\,695$ contaminantes + $1\,401\,685$ KN = $2\,999\,380$, exactamente el total del split (sección 9.7). Y aplicando el reparto 90/5/5 clase por clase sobre esos conteos salen los $\sim 80\text{k}$ contaminantes de test; con la columna de transientes únicos saldría la mitad.

5. Receta de error fotométrico

Es el corazón cuantitativo del proyecto y vive en un único módulo, `src/kilonova/photometry/roman_noise.py`, para que pipeline y notebooks no se desincronicen.

5.1. La ecuación CCD

Todo parte de la ecuación estándar de CCD (Howell 1989, eq. 14) en espacio de **electrones**:

$$\sigma^2(F) = \underbrace{F_{\text{source}}}_{\text{Poisson de la fuente}} + \underbrace{\text{NEA} (B_{\text{sky}} + B_{\text{thermal}} + B_{\text{dark}} + \sigma_{\text{read}}^2)}_{\text{fondo, por pixel} \times \text{pixeles efectivos}} \quad (6)$$

El primer término es la varianza de Poisson de N electrones (que es N). El segundo es el fondo por píxel multiplicado por el número efectivo de píxeles de la fotometría PSF: el **NEA** (*Noise Equivalent Area*, King 1983; Naylor 1998), que es el n_{pix} efectivo de una extracción óptima con pesos de PSF.

5.2. Flujo de la fuente y magnitud límite

Con A el área colectora (`galsim.roman.collecting_area`) y ZP el punto cero AB de la banda:

$$F_{\text{source}} = t_{\text{exp}} A 10^{-0.4(m_{\text{true}} - \text{ZP})} \quad [e^-] \quad (7)$$

$$\sigma_F = \sqrt{F_{\text{source}} + \mathcal{N}_{\text{band}}}, \quad \mathcal{N}_{\text{band}} \equiv \text{NEA} (B_{\text{sky}} + B_{\text{th}} + B_{\text{dark}} + \sigma_{\text{read}}^2) \quad (8)$$

$$S/N = F_{\text{source}}/\sigma_F, \quad \text{detección si } S/N \geq 5 \quad (9)$$

$$m_{\text{lim}, 5\sigma} = \text{ZP} - 2.5 \log_{10} \left(\frac{5 \sigma_F}{t_{\text{exp}} A} \right) \quad (10)$$

$\mathcal{N}_{\text{band}}$ es constante por (banda, tier): no depende del objeto, así que se calcula una vez en `build_tier_constants`.

El error de magnitud sale de la propagación logarítmica:

$$\sigma_m = \frac{2.5}{\ln 10} \frac{\sigma_F}{F} = \frac{1.0857}{S/N} \quad (11)$$

Y la magnitud *observada* es una realización ruidosa en **espacio de flujo** (no de magnitud, que sería sesgado y explotaría con flujos negativos):

$$F_{\text{obs}} = F_{\text{true}} + \mathcal{G}(0, \sigma_F), \quad m_{\text{obs}} = \text{ZP} - 2.5 \log_{10}(F_{\text{obs}}/(t_{\text{exp}} A)) \quad (12)$$

Si $F_{\text{obs}} \leq 0$ se reporta $m_{\text{lim}, 5\sigma}$ en su lugar.

5.3. Ruido de lectura *up-the-ramp*

Roman lee el detector de forma no destructiva cada $t_{\text{frame}} = 3.04$ s; con $n = t_{\text{exp}}/t_{\text{frame}}$ lecturas (Rose et al. 2025, eq. 9):

$$\sigma_{\text{read}} = \sqrt{\sigma_{\text{floor}}^2 + \sigma^2 \frac{n-1}{n(n+1)}} \quad \sigma_{\text{floor}}^2 = 25 \, e^{-2}, \quad \sigma^2 = 12 \cdot 16^2 = 3072 \, e^{-2} \quad (13)$$

Pregunta segura de examen. ¿Por qué el denominador es $n(n+1)$ y no $(n+1)$? Porque el ajuste de la rampa promedia n lecturas: la varianza del *slope* estimado cae como $1/n$ adicional. Con $(n+1)$ saldrían $55.46 e^-$, un factor 9 de más. Verificación numérica: $t_{\text{exp}} = 900 \text{ s} \Rightarrow n = 296 \Rightarrow \sigma_{\text{read}} = 5.942 e^-$, que es exactamente el 5.942 de la Tabla 3 del paper. **Cuidado al citarlo:** esos 900 s son la exposición *de la Tabla 3*, no la nuestra — la F184 deep del pipeline es de 1636 s y da $5.540 e^-$. Son dos números distintos y hay que decir cuál es cuál.

Consecuencia contraintuitiva: σ_{read} *decrece* con t_{exp} y satura en el piso $\sqrt{25} = 5 e^-$. Medidos: $5.540 e^-$ (F184 deep, 1636 s), $7.491 e^-$ (H158, 294 s), $12.870 e^-$ (R062 wide, 60 s). Exposiciones cortas son *más* ruidosas de lectura que las largas.

5.4. Fondo: cielo, térmico, dark

Todo sale de `galsim.roman` evaluado en el centro del campo del tier:

$$B_{\text{sky}} = \text{getSkyLevel}(\text{banda}, \text{pos}, t_{\text{exp}}) \times \text{pixel_scale}^2 \quad [e^-/\text{pix}] \quad (14)$$

$$B_{\text{thermal}} = \text{thermal_backgrounds}[\text{banda}] \cdot t_{\text{exp}}, \quad B_{\text{dark}} = \text{dark_current} \cdot t_{\text{exp}} \quad (15)$$

Ojo con las unidades: `getSkyLevel` devuelve e^-/arcsec^2 ; hay que multiplicar por `pixel_scale`² *antes* de sumar los términos por píxel.

5.5. NEA tabulada, no calculada

NEA [pix]	R062	Z087	Y106	J129	H158	F184
	5.575	6.695	7.895	9.210	11.140	16.335

Son los valores de la Tabla 3 de Rose et al. (2025): mediana entre el mejor y el peor punto del FOV, tomados de la PSF del sitio técnico de la WFI. **No** se usa `roman.getPSF` porque la PSF analítica del Ciclo 9 de `galsim` sobrestima el NEA un 15–28 % en las bandas azules (R/Z/Y); H y F sí coinciden. Es independiente del tier y del tiempo de exposición.

5.6. Jitter del punto cero

El ZP efectivo depende de la posición en el FOV. Se modela (Rose et al. 2025, eq. 8) como un jitter gaussiano de $\sigma_{\text{ZP}} = 0.15 \text{ mag}$, sorteado **una vez por (banda, visita)** y fijo por objeto. Se usan dos flujos de números aleatorios independientes (`spawn` de una `SeedSequence`) para el jitter y para la realización de ruido, de modo que no queden correlacionados.

5.7. La misma ecuación en el lenguaje de SNANA (Hourglass)

Hourglass entrega flujos en unidades `fluxcal` con $\text{ZP}_{\text{SNANA}} = 27.5$: $m = 27.5 - 2.5 \log_{10}(\text{fluxcal})$. Traducida a esas unidades, la Ec. (6) es

$$F_{\text{sim}} = \text{fluxcal} \cdot 10^{0.4(m - \text{sim_mag_obs})}, \quad \alpha \equiv 10^{(27.5 - \text{zp})/2.5} \quad (16)$$

$$\sigma_{\text{pred}}^2 = \underbrace{\alpha F_{\text{sim}}}_{\text{fuente}} + \underbrace{\text{NEA}(\sigma_{\text{sky}}^2 + \sigma_{\text{read}}^2) \alpha^2}_{\text{fondo}} \quad (17)$$

Pregunta clásica: ¿por qué un término lleva α y el otro α^2 ?

Los dos llevan α^2 de propagación de unidades; en el de fuente uno de los dos factores se cancela solo, porque la varianza de Poisson es **lineal** en las cuentas. Con $\mathbf{fcal} = \alpha F_e$:

$$\text{Var}_{\mathbf{fcal}} = \alpha^2 \text{Var}_e = \alpha^2 F_e = \alpha^2 \frac{F_{\mathbf{fcal}}}{\alpha} = \alpha F_{\mathbf{fcal}}$$

El fondo no cancela nada porque ya nace *siendo* una varianza en $[e^{-2}/\text{pix}]$: ahí α^2 es propagación pura $[e^{-2}] \rightarrow [\mathbf{fcal}^2]$. No interviene ningún factor de Fano — para un Poisson vale 1 por definición, y α no es eso sino un cambio de unidades.

Hourglass suma además un término de *host surface brightness* dentro de $\mathbf{fluxcal_err}$ que no es desagregable. Por eso la validación reproduce $\mathbf{fluxcal_err}$ con razón mediana ≈ 0.986 , y el p05 baja en F/H deep, donde el host domina.

5.8. Referencias de la receta

Howell (1989) *PASP* 101, 616 (ecuación CCD) · King (1983) *PASP* 95, 163 (NEA) · Naylor (1998) *MNRAS* 296, 339 (extracción óptima) · Fano (1947) · Kessler et al. (2009) *PASP* 121, 1028 (SNANA) · Hounsell et al. (2018) *ApJ* 867, 23 · Rose et al. (2025) *ApJ* 988, 65 (Hourglass; eqs. 8–10 y Tabla 3).

6. La grilla LANL de kilonovas

6.1. Parámetros

900 simulaciones radiativas únicas ($\mathbf{simulation_id}$), cada una con **dos componentes de eyecta** (dinámica y viento), cada componente con su masa y su velocidad:

parámetro	significado	valores
$\mathbf{run_type}$	morfología (dyn toroidal + viento)	TP (peanut), TS (esférico)
$\mathbf{wind}$	composición del viento	wind1 ($Y_e=0.27$), wind2 ($Y_e=0.37$)
$\mathbf{mass_dynamical}$	masa eyecta dinámica	0.001, 0.003, 0.01, 0.03, 0.1 $M_\odot$
$\mathbf{velocity_dynamical}$	velocidad dinámica	0.05, 0.15, 0.3 c
$\mathbf{mass_wind}$	masa del viento	0.001, 0.003, 0.01, 0.03, 0.1 $M_\odot$
$\mathbf{velocity_wind}$	velocidad del viento	0.05, 0.15, 0.3 c
$\mathbf{angle_index}$	bin de ángulo de visión	0–53 (54 bins uniformes en $\cos\theta$)

La cuenta de las 900 simulaciones cierra exactamente:

$$\underbrace{2}_{\text{morfología del viento}} \times \underbrace{2}_{\text{composición del viento}} \times \underbrace{5}_{m_{\text{dyn}}} \times \underbrace{3}_{v_{\text{dyn}}} \times \underbrace{5}_{m_{\text{wind}}} \times \underbrace{3}_{v_{\text{wind}}} = 900 \quad (18)$$

y $900 \times 54 = \mathbf{48\,600}$ realizaciones físicas distintas — exactamente la marginalización uniforme (sin pesos) que promedia Chase et al. (2021).

Qué son $\mathbf{run_type}$ y $\mathbf{wind}$ en física. Los dos ejes “categóricos” de la grilla son morfología y composición de la eyecta (Tabla 2 de Chase+21):

- La **eyecta dinámica** es siempre *toroidal* y con $Y_e = 0.04$ fijo (muy neutronizada $\Rightarrow$ produce lantánidos $\Rightarrow$ opacidad alta y emisión roja). No es un eje de la grilla: es una constante del set.
- $\mathbf{run_type} \in \{\text{TP, TS}\}$ codifica la **morfología del viento**: T(oroidal dinámica) + viento en forma de “P(eanut)” o “S(férico)”. Cambia el ángulo sólido que cada componente cubre y, por tanto, cuánta eyecta roja se interpone en la línea de visión.

- `wind` $\in \{\text{wind1}, \text{wind2}\}$ codifica la **composición del viento**: $Y_e = 0.27$ y $Y_e = 0.37$. El viento es menos neutronizado que la eyecta dinámica, así que es lantánido-pobre: opacidad menor, emisión más azul y más rápida.

Ésa es la razón física de que el ángulo importe tanto: mirando por el polo se ve el viento azul casi sin obstrucción; mirando por el ecuador, el toro dinámico rojo tapa el viento. Es el mismo efecto que hace que las 54 realizaciones angulares de *una misma* simulación puedan diferir en varias magnitudes.

6.2. Ejes continuos: tiempo y longitud de onda

Los espectros están tabulados en reposo sobre dos rejillas logarítmicas. Números medidos directamente sobre `lanl_spectra.parquet` (3 301 884 filas = una por (simulación, tiempo, ángulo)):

eje	rango	muestreo
tiempo post-merger (reposo)	$t_{\min} = 0.125 \text{ d} = \mathbf{3 \text{ h}}$ $t_{\max} = 64 \text{ d}$	73 nodos logarítmicos, razón $r = 512^{1/72} = 1.0905$
longitud de onda (reposo)	$\lambda_{\min} = 1002.35 \text{ \AA} = 0.100 \text{ }\mu\text{m}$ $\lambda_{\max} = 127\,695 \text{ \AA} = 12.77 \text{ }\mu\text{m}$	1024 nodos logarítmicos, razón 1.00475 ($R \approx 211$)

Detalles importantes de esas rejillas:

- $t_{\min} = 0.125 \text{ d}$ es un **corte duro**: no existe ningún espectro antes de las 3 horas post-merger. En el observador eso son $0.125(1+z)$ d, así que a $z = 0.3$ la primera información disponible es a $\sim 3.9 \text{ h}$. Sesga las bandas azules (que es donde la KN es brillante justo al principio) hacia z bajo, y el propio Chase+21 lo declara como limitación, no es un artefacto nuestro.
- $t_{\max} = 64 \text{ d}$ es el **techo del set, pero no el de cada simulación**: solo **100 de las 900** simulaciones llegan a 64 d. Por simulación hay entre **63 y 73** nodos temporales (mediana 67) y el último tiempo va de **26.9 d a 64 d**; los cortes más frecuentes están en 26.9, 34.9, 38.1, 49.3 y 53.8 d. Los modelos más débiles y rápidos son los que terminan antes. Para nuestro problema es irrelevante: la ventana temprana cubre 4 épocas $\times 5 \text{ d} = 20 \text{ d}$ observador, dentro del rango de *todas* las simulaciones.
- El rango en λ (0.1–12.8 μm) cubre holgadamente los seis filtros Roman (0.48–2.00 μm) incluso redshiftado: a $z = 1$, F184 muestrea $\sim 0.92 \text{ }\mu\text{m}$ en reposo, muy dentro de la rejilla. La resolución $R \approx 211$ es más que suficiente para fotometría de banda ancha.
- Las unidades del flujo almacenado son $\text{erg s}^{-1} \text{ cm}^{-2} \text{ \AA}^{-1}$, a **10 pc**, en convención *isotrópico-equivalente* (ver el factor 54 más abajo). Todo esto está escrito en la metadata del parquet, no hay que recordarlo de memoria.
- Interpolación: para un tiempo observador t_{obs} y un redshift z se entra en la rejilla con $t_{\text{rest}} = t_{\text{obs}}/(1+z)$. Y no es interpolación sino **vecino más cercano** (`nearest_time_index`), en tiempo y en fase. En λ sí hay control de cobertura y una banda no cubierta devuelve NaN; en *tiempo* no lo hay: pasado el último nodo de la simulación se queda **pegado** a ese último espectro en vez de excluirse. Afecta poco porque la ventana son 4 épocas desde la primera detección y las simulaciones llegan a 26.9 d como mínimo, pero es lo que hace el código.

6.3. El bug grande: el factor angular 54

Los ficheros `_spec_` de LANL guardan el flujo **por bin angular**, no el equivalente isotrópico. Un observador situado en el bin k ve una fuente cuya luminosidad aparente es la de ese bin repartida por toda la esfera:

$$\text{factor} = \frac{4\pi}{\Delta\Omega_{\text{bin}}} = n_{\text{angles}} = 54 \implies \Delta m = -2.5 \log_{10} 54 = \mathbf{-4.331 \text{ mag}} \quad (19)$$

Sin él **toda** kilonova sintética salía 4.33 mag demasiado débil (llegábamos a $z = 0.24$ donde Chase+21, con la misma grilla, llega a $z \sim 1$). Verificación: la integral bolométrica del espectro reproduce $L_{\text{bol}}/(4\pi(10\text{ pc})^2)$ de los ficheros `_lums_` con razón 1.0000 en todas las fases (los espectros están a 10 pc y se leían bien), y los bloques de banda de `_lums_` son L_ν , que encajan con `_mags_` vía $f_\nu = L_\nu/(4\pi(10\text{ pc})^2)$. La convención queda registrada en la metadata del parquet (`flux_convention`): **no aplicar el factor dos veces**.

6.4. Del espectro en reposo a la magnitud AB observada

Trampa de la que es fácil caer (y estas notas cayeron). El repositorio tiene *dos* caminos de fotometría de kilonovas y solo uno genera los datos de entrenamiento. El módulo `simulation/extinction.py` documenta una cadena preciosa con extinción de host, extinción galáctica y reglas de parada — y **no es la que produce `kn_windows_*.parquet`**. Es el CLI `kn-extinguish`, una rama paralela que escribe “fotometría extinguida” y muere ahí.

La cadena real del CLI `kn-kilonova-windows`, que es la que importa (`early_windows.kn_model_from_spectra` → `photometry/spectra.py`):

$$\text{espectro rest} \xrightarrow[\text{specutils}]{\text{clip}(f_\lambda \geq 0)} \xrightarrow[\text{redshift}]{(10\text{ pc}/d_L)^2/(1+z), \text{ Planck18}} \xrightarrow[\text{galsim.roman}]{\text{bandpasses}} m_{\text{AB}}^{\text{Roman}} \quad (20)$$

Sin polvo, con galsim y no pyphot, y sin reglas de parada. De `extinction.py` el CLI importa exactamente una cosa: `build_redshift_grid`. Consecuencia científica, que hay que declarar como sistemático: las KN del set no llevan extinción ninguna, mientras que los contaminantes llevan la del host dentro de su modelo SED (ver el cuadro de la sección 4.2). Es la única receta que *no* se comparte entre las dos ramas.

¿Y ese $1/(1+z)$ extra no contradice “solo dos factores $(1+z)$ ”? No. Los dos $(1+z)$ físicos están dentro de d_L y no se aplican otra vez. El tercero es el **jacobiano** $d\lambda_{\text{obs}}/d\lambda_{\text{rest}} = (1+z)$: `specutils` solo estira el eje de longitud de onda, así que la *densidad* f_λ tiene que llevar el inverso para que $\int f_\lambda d\lambda$ siga dando $L/(4\pi d_L^2)$. Fotométricamente es el término de K-corrección; quitarlo “por duplicado” deja toda fuente $2.5 \log_{10}(1+z)$ demasiado brillante.

Cosmología para comparar con Chase+21: Λ CDM plana con $H_0 = 67.4$, $\Omega_m = 0.315$, $\Omega_\Lambda = 0.685$ (Planck 2020).

Lo que sí vive en `extinction.py` (para la otra rama). Cadena `rest` → `F99( $A_V^{\text{host}}, R_V^{\text{host}}$ )` → `redshift` → `dimming` → `F99( $A_V^{\text{MW}} = 3.1 E(B-V)$ )` → `pyphot`, con el polvo sorteado una vez por `redshift` y $E(B-V)$ del mapa SFD, más dos reglas de parada para no evaluar la grilla entera: **STOP-A** (rompe el bucle de z tras `z_patience` redshifts consecutivos sin ninguna detección) y **STOP-LC** (trunca el eje temporal cuando *todas* las bandas acumulan `lc_patience` pasos decrecientes no detectados). Nada de esto toca al clasificador.

6.5. Muestreo de ventanas KN

Una “KN observada” es una realización de (simulación, ángulo, offset de explosión, z , paridad, ruido):

- **Grilla de z :** logarítmica. Por defecto del CLI $z \in [0.01, 1]$ con 50 nodos y 200 realizaciones/nodo; la corrida que entrenó el modelo usó 100 nodos en $[0.02, 1]$ y **10 000 realizaciones por nodo**.
- **Offset de explosión:** $t_0 \sim U[0, 5]$ d, días observador entre el merger y la primera visita. Las visitas de una KN son $t_0 + 5 \cdot \{0, \dots, 7\}$ (8 visitas: margen para encontrar la 1.^a detección más 4 épocas).
- **Paridad de cadencia:** sorteada aparte, ver más abajo.

- Las realizaciones se comparten entre tiers: *la misma* KN se observa en deep y en wide.
- Las realizaciones nunca detectadas **no se escriben**. Consecuencia importante para cualquier estadística: el denominador es el número de realizaciones sorteadas, no el número de filas; los ceros del mapa son los objetos ausentes.
- `object_id = {simulation_id}_{angle_index}_{offset:.4f}_{z:.4f}_{parity}_{noise_id}` — los parámetros no son columnas, van codificados en el id (y el `simulation_id` va primero porque el split anti-fuga lo lee de ahí).

La fuga de la paridad de cadencia (muy preguntable). El patrón de bandas se repite cada 2 visitas, así que la fase del merger dentro del ciclo tiene *dos* grados de libertad: el retardo hasta la primera visita (t_0) y la **paridad** de esa visita. Como la grilla KN se construye como $t_0 + 5 \cdot \text{arange}(N)$, su índice 0 era *siempre* par. Y como la KN es rápida y casi siempre se detecta en la primera visita, eso imprimía la fase de la cadencia en la clase KN: 92 % par frente a 29 % en los contaminantes, y **la máscara de bandas sola clasificaba al 80.7 %**. En un survey real la grilla es fija en tiempo absoluto y el merger cae uniforme en el ciclo de 10 d, o sea paridad 50/50 e independiente del retardo. Solución: sortear la paridad.

Y ojo con el remate: en la población que *sobrevive* la paridad no sale 50/50 sino 48/52 (deep) y 45/55 (wide). No es que el arreglo falle: la paridad también decide si esa visita lleva las bandas rojas, y eso cambia la *detectabilidad*. Ese residuo es físico y legítimo — en un survey real pasaría igual — a diferencia del 92 % original, que era una huella del generador.

6.6. Validación contra Chase et al. (2021)

Chase+21 (arXiv:2105.12268) usa esta misma grilla, así que es la prueba externa. Fracción de KN detectables, contornos leídos de la figura de Roman/H:

contorno	Chase+21	nuestro (deep)	nuestro (wide)
0.95	~0.09	~0.095	0.088
0.50	0.22	0.215	0.201
0.05	~0.48	0.477	0.454
corte (pico del contorno 0.5)		paper	nuestro
$m_{\text{dyn}} = 0.1 M_{\odot}$		0.31	~0.31
$m_{\text{wind}} = 0.1 M_{\odot}$		0.37	~0.365
$m_{\text{dyn}} = 0.001 M_{\odot}$		0.16	~0.147

Coinciden dentro de la tolerancia de ± 0.02 . La forma también reproduce la Fig. 3 del paper: pico en $t \sim 1$ d en las bandas azules y ~ 3 d en F184, alcance creciente hacia el rojo. El criterio del paper es $m_{\text{true}} < m_{\text{lim}}$ con su Tabla 1 fija; el nuestro es $S/N \geq 5$ con la receta de ruido. En *deep* el nuestro llega más lejos ($z_{50\%} = 0.49$ en F184) porque el tier deep es 1–2 mag más profundo por visita que lo que tabula el paper (F184: 27.0 vs 24.9); en *wide*, donde las profundidades coinciden, los dos criterios dan casi lo mismo — que es la comprobación cruzada interesante.

7. La ventana temprana

Regla única, idéntica para KN y contaminantes (`epochs_from_first_detection`):

1. Se calcula S/N en todas las (visita, banda) que la cadencia observa.
2. **Primera detección** = la visita más temprana con $S/N \geq 5$ en *cualquier* banda. Si no hay ninguna, el objeto se descarta (nunca entraría en un pipeline de transientes).
3. Se toman **hasta 4 épocas consecutivas** desde ahí (`NUMBER_OF_EPOCHS = 4`). Una visita sin detección igual cuenta como época. “Hasta”: si el modelo se acaba antes salen menos, y de hecho ~0.7 % de los contaminantes tienen 1–3 épocas. Las KN casi siempre tienen las 4, porque su grilla son 8 visitas.

4. Eje temporal: `days_since_detection = tepoch - t1a detección`. El MJD absoluto nunca se usa. El modelo ve **3 épocas** (EPOCHS_PER_WINDOW= 3); la 4.^a se guarda como *buffer* para la augmentación de desplazamiento.

Cuántos tokens hay, exactamente. El vocabulario tiene 6 bandas, pero **cada tier observa solo 5** (deep ZYJHF, wide RZYJH): el vocabulario es la *unión* de los dos tiers y ninguna secuencia individual las lleva todas. Así que son 3 épocas × 5 bandas = **15** tokens como máximo, más [CLS] y [Z]: **≤17 posiciones**. (Se comprueba en el parquet: 20 filas por objeto = 4 épocas × 5 bandas.)

Esquema de fila del parquet (14 columnas): `object_id`, `gentype`, `label`, `z_CMB`, `epoch`, `days_since_detection`, `band`, `observed`, `mag_true`, `mag_observed`, `mag_err`, `snr`, `detected`, `mag_limit_5sigma`. Los `early_windows_*` de contaminantes llevan una 15.^a, `snana_id`, que *no* es un id de plantilla SED (solo tiene 33 valores, uno por healpix) y por tanto no sirve para el split.

8. Tokens

8.1. Un token = una medición (objeto, visita, banda)

campo	qué es
<code>delta_time</code>	días desde la época ancla de la ventana (float, sin normalizar; lo codifica Time2Vec)
<code>band_index</code>	0–5
<code>token_type_index</code>	d=0 detección, u=1 upper limit, n=2 no observado
<code>magnitude</code>	$(m_{\text{obs}} - \mu_m)/\sigma_m$, o 0.0 con <code>magnitude_mask=0</code>
<code>sigma_magnitude</code>	$(\sigma_{m,\text{obs}} - \mu_\sigma)/\sigma_\sigma$, o 0.0 con <code>sigma_mask=0</code>
<code>magnitude_mask</code> , <code>sigma_mask</code>	1.0 si el valor es real, 0.0 si es relleno

Los tres tipos de token codifican **tres niveles de información** distintos:

- **d** — detección (`detected`, o $S/N \geq 5$ si falta el flag): magnitud real + σ_m real.
- **u** — observado pero no detectado: la magnitud es el **límite 5σ de esa visita**, Ec. (10), sin σ . Nunca una profundidad fija de survey: la profundidad realmente alcanzada *es* el contenido informativo de la no-detección.
- **n** — esa banda no se observó esa noche: magnitud y σ enmascaradas, pero el token conserva (Δt , banda). La cadencia de Roman es determinista, así que “no se miró” es información distinta de “se miró y no había nada”.

8.2. Normalización global

Nunca por objeto — el brillo aparente *es* señal. Estadísticas ajustadas **solo sobre los tokens de detección (d) del split de train** — los límites superiores no entran en μ ni en σ , aunque después se normalicen con ellos — y guardadas en `normalization.json`:

$$\mu_m = 24.9204, \quad \sigma_m = 1.6660, \quad \mu_\sigma = 0.09270, \quad \sigma_\sigma = 0.06418$$

8.3. Tokens globales y regímenes de redshift

- [CLS]: token aprendido; su salida final alimenta la cabeza de clasificación.
- [Z]: si hay z , se proyecta linealmente a d_{model} ; si no, se usa un token aprendido `no_redshift_token`.
- **Dropout de redshift** = 0.50 en train: la mitad de las veces se oculta el z verdadero, para que el mismo modelo sirva con y sin z . En validación se miden los dos regímenes por separado (`val_acc_z` vs `val_acc_noz`).

8.4. Augmentaciones

Son **tres**, todas estocásticas en el dataloader (se re-sortean cada vez que el objeto entra en un batch, no son un split fijo) y todas apagadas en validación y test:

1. **Truncamiento aleatorio de prefijo.** Cada ejemplo se recorta a **1, 2 o 3 épocas** con probabilidad uniforme (`rng.integers(1, 4)`). **Ésta es la augmentación clave:** es lo único que hace que el modelo funcione con una sola época, y sin ella la matriz de degradación de la sección 10 no tendría sentido.
2. **Desplazamiento de ventana**, probabilidad 0.20: se descarta la época 1 y se re-ancla Δt en la 2. Solo para objetos elegibles (con detección real en la segunda visita) — de ahí que se guarde la 4.^a época. Simula el $\sim 20\%$ de eventos que la fase de explosión uniforme captura ya con ≥ 4 d de edad.
3. **Dropout de redshift**, probabilidad 0.50 (sección anterior).

Las tres se aplican igual a KN y contaminantes, para que el régimen sea simétrico entre clases.

8.5. Sin token de modo deep/wide

Decisión de diseño con justificación explícita: lo que un *embedding* de modo llevaría — la profundidad realmente alcanzada — ya está en los tokens de forma continua (σ_m para detecciones, $m_{\text{lim},5\sigma}$ para no-detecciones). Añadirlo grabaría la rejilla $\text{banda} \times \text{tier}$ en el encoder como una huella dactilar del simulador, ampliando la brecha de dominio con los datos reales de Roman.

9. El transformer

Fichero `training/model.py`: encoder-only tipo *set transformer*. Los tokens son un **conjunto indexado por (tiempo, banda)**, no una secuencia ordenada: el tiempo entra como *feature continua dentro del token*, nunca como positional encoding.

9.1. Time2Vec

$$\tau = \frac{\Delta t}{\Delta t_{\text{scale}}}, \quad \Delta t_{\text{scale}} = 5 \text{ d}; \quad \text{t2v}(\Delta t) = W \left[\underbrace{w_0 \tau + b_0}_{\text{lineal}}, \underbrace{\sin(w_i \tau + b_i)}_{i=1 \dots K} \right] + b_W \in \mathbb{R}^{d_{\text{model}}} \quad (21)$$

(W y b_W son la `nn.Linear` final; el bias existe y hay que contarlo.) El **término lineal es el caballo de batalla**: es lo que da a la atención un eje temporal *continuo* del que después se extrae la *tasa de decaimiento* $\Delta m / \Delta t$, que es la física discriminante (KN: días; SN: semanas). El mecanismo exacto — que **no** es una resta dentro del softmax — está en la sección 9.3.

En el modelo actual $K = 0$ (`TIME2VEC_FREQUENCIES = 0`): con 3 épocas sobre ~ 15 d no hay periodicidad resoluble ni física, y los transientes no son pulsadores. Los pesos w_i se inicializan en `linspace(0.5, 3.0)` (periodos ~ 10 – 60 d) si se activan.

Consecuencia de $K = 0$, que hay que saber decir. Sin sinusoides el encoding temporal colapsa a un **mapa afín de rango 1**:

$$\text{t2v}(\Delta t) = \mathbf{u} (w_0 \tau + b_0) + b_W, \quad \mathbf{u} \in \mathbb{R}^{192} \text{ fijo: la única columna de } W \quad (22)$$

Es decir: **el tiempo ocupa una sola dirección** del espacio de 192 dimensiones y su coordenada a lo largo de ella es lineal en Δt . No es un código rico, es una regla numerada — y es exactamente lo que hace falta para que el logit de atención salga bilineal en los dos tiempos (Ec. (28)).

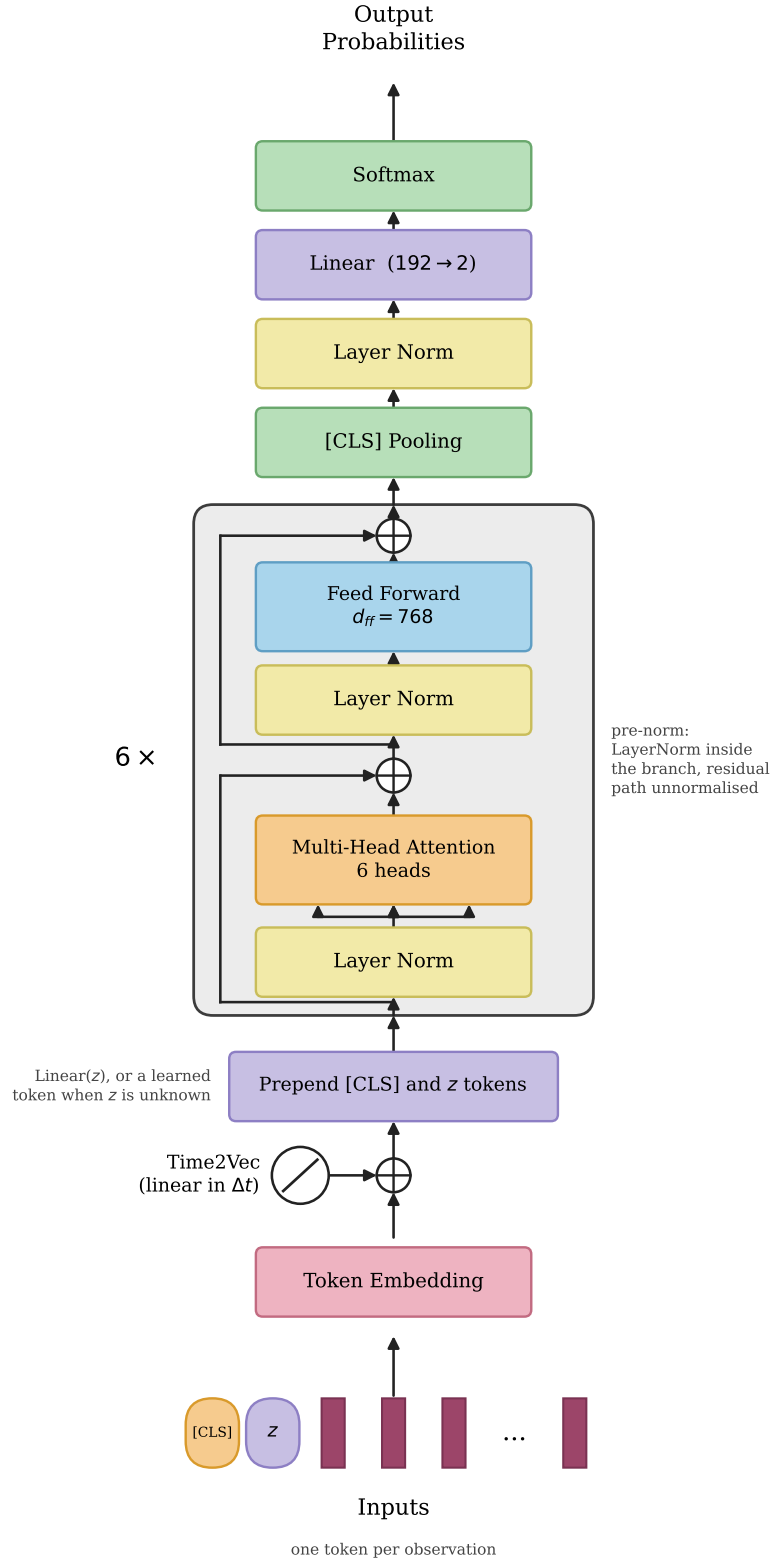


Figura 1: Paso *forward* de KilonovaTransformer (training/model.py), generado por training/plot_architecture_diagram.py. El bloque encoder es **pre-norm** (sección 9.4); el encoding temporal se dibuja como una rampa recta y no como una senoide porque TIME2VEC_FREQUENCIES= 0, así que solo sobrevive el término lineal en Δt .

9.2. Fusión del token

$$\text{content} = \text{LayerNorm}\left(\text{Linear}\left[e_{\text{band}} \parallel e_{\text{type}} \parallel (m, \sigma_m, \text{mask}_m, \text{mask}_\sigma)\right]\right), \quad x = \text{content} + t2v(\Delta t) \quad (23)$$

El tiempo se **suma**, el resto se concatena y se proyecta. Dimensiones de los embeddings: $d_{\text{band}} = 6$, $d_{\text{type}} = 3$ — una tabla de N filas seguida de una capa lineal tiene rango efectivo $\leq \min(N, D)$, así que $D = N$ es la elección más pequeña sin pérdida.

Secuencia de entrada: $[\text{CLS}, \text{Z}, x_1, \dots, x_N]$, sobre la que se aplica un `input_dropout global` — también a CLS y Z, no solo a los tokens de medición — más una máscara de padding. Formas reales para un batch de 512:

tensor	forma	origen
token_vectors	$(512, \leq 15, 192)$	TokenEmbedding
cls_token, redshift_token	$(512, 1, 192)$ c/u	GlobalTokens
sequence	$(512, \leq 17, 192)$	cat + input_dropout
attention_mask	$(512, 1, 1, \leq 17)$	~padding_mask + dos True

La máscara es de *keys*, no de *queries*. Su forma hace broadcast sobre las filas,

$$(B, 1, 1, L) \longrightarrow (B, h, L_{\text{query}}, L_{\text{key}}),$$

o sea que enmascara **columnas**. Una posición de padding sigue produciendo un vector de salida (basura), pero nadie lo lee — solo se usa `encoded[:, 0]`. Y como CLS y Z son siempre keys válidas, **ninguna fila del softmax queda entera enmascarada**, que es la situación que daría NaN; el -9×10^{15} en vez de $-\infty$ es el segundo cinturón de seguridad.

No hay positional encoding, y la máscara no distingue posiciones. Por tanto el encoder es *equivariante a permutaciones* de los tokens y la salida de CLS es *invariante*. Eso es lo que hace literal la etiqueta “set transformer”: barajar las filas del parquet no cambia el logit, y es comprobable en tres líneas.

9.3. Atención, paso a paso

Es el mecanismo central y conviene recorrerlo sin saltarse ningún paso. Todas las formas de abajo son las del modelo entrenado: $d_{\text{model}} = 192$, $h = 6$ cabezas, $d_{\text{head}} = 192/6 = \mathbf{32}$, $L \leq 17$ posiciones.

Paso 1 — qué son *query*, *key* y *value*. Cada posición emite **tres** vectores distintos derivados del mismo x . La analogía correcta es una búsqueda en un diccionario blando:

vector	rol	pregunta que responde
<i>query</i> q_i	lo que la posición i busca	¿qué información me falta?
<i>key</i> k_j	el índice con que j se anuncia	¿de qué trato yo?
<i>value</i> v_j	el contenido que j entrega	¿qué me llevo si la atiendo?

La compatibilidad entre i y j es el producto punto $q_i \cdot k_j$; el softmax sobre j lo convierte en pesos y la salida de i es la mezcla de los v_j .

Por qué *key* y *value* son vectores distintos — el punto que casi siempre se explica mal: el *criterio de selección* y el *contenido transportado* son independientes y se aprenden por separado. Aquí un token puede anunciarse por su *key* como “soy banda H a $\Delta t = +5$ d” (criterio temporal y

de banda) mientras su value transporta “magnitud normalizada -0.8 , σ_m pequeño” (contenido fotométrico). Si key y value fueran el mismo vector no se podría a la vez *seleccionar por tiempo* y *leer magnitud*.

El [CLS] es el caso límite: **no tiene contenido propio** — es un `nn.Parameter` aprendido, idéntico para todos los objetos — así que su único trabajo es emitir una query. Lo que esa query aprendió a pedir es la evidencia discriminante; lo que se lleva es la suma ponderada de los valores de las mediciones. Por eso al final solo se lee la posición 0.

Paso 2 — de dónde salen: la proyección fusionada. El código no usa tres matrices W_Q, W_K, W_V de 192×192 sino **una sola** de 192×576 , y la corta después:

```
qkv = self.qkv_projection(x)           # (B, L, 576)
qkv = qkv.reshape(B, L, num_heads, 3 * head_dim) # (B, L, 6, 96)
qkv = qkv.permute(0, 2, 1, 3)         # (B, 6, L, 96)
query, key, value = qkv.chunk(3, dim=-1) # 3 x (B, 6, L, 32)
```

El layout de las 576 filas es `[h0_q(32) h0_k(32) h0_v(32) | h1_q(32) ...]`: primero por *cabeza*, después en q/k/v. Es matemáticamente idéntico a las tres matrices separadas — una permutación de columnas de una única matriz aprendida — y ahorra dos GEMM.

Trampa de portabilidad. `nn.MultiheadAttention` de PyTorch usa el orden *contrario* en su `in_proj_weight`: `[todos_q(192); todos_k(192); todos_v(192)]`. Cambiar a la implementación nativa (o a `F.scaled_dot_product_attention`) sin permutar las filas carga los pesos en silencio y devuelve basura. El `assert d_model % num_heads == 0` es lo que garantiza que la partición en cabezas sea exacta.

Paso 3 — el producto punto escalado. Con LN la LayerNorm de la rama (pre-norm, sección siguiente):

$$\left[q_i^{(h)} \parallel k_i^{(h)} \parallel v_i^{(h)} \right]_{h=1}^6 = W_{qkv} \text{LN}(x_i) + b_{qkv}, \quad W_{qkv} \in \mathbb{R}^{576 \times 192} \quad (24)$$

$$\ell_{ij}^{(h)} = \frac{q_i^{(h)} \cdot k_j^{(h)}}{\sqrt{d_{\text{head}}}}, \quad \ell_{ij}^{(h)} \leftarrow -9 \times 10^{15} \text{ si } j \text{ es padding} \quad (25)$$

$$\alpha_{ij}^{(h)} = \text{softmax}_j(\ell_{ij}^{(h)}), \quad \sum_j \alpha_{ij}^{(h)} = 1 \quad (26)$$

Por qué se divide por $\sqrt{d_{\text{head}}}$. $q \cdot k$ es una suma de 32 productos; si las componentes son $\sim \mathcal{N}(0, 1)$ independientes, esa suma tiene varianza 32 y desviación ≈ 5.7 . Con 17 keys, logits de esa escala dejan el softmax prácticamente *one-hot* ya en la inicialización: el gradiente se anula en 16 de las 17 entradas y la capa no aprende a mezclar. Dividir por $\sqrt{32}$ devuelve varianza 1 y mantiene el softmax en su régimen útil. Es normalización de varianza, no una constante de conveniencia.

Cómo se lee la matriz 17×17 . La fila i es “a quién mira i ” y suma 1; la columna j es “quién mira a j ” y **no** suma 1. Al plotear los mapas de `get_attention_maps`: la fila 0 es la de [CLS], que es la interesante, y las filas de posiciones *padded* son ruido — el método no las recorta.

Paso 4 — multi-head: por qué 6 y no 1. Un solo softmax por posición sólo puede expresar **un** patrón de enrutamiento: si [CLS] quisiera a la vez “el token más tardío de F184” y “el más temprano de Z087”, una cabeza sola tendría que promediarlos y perdería los dos. Seis cabezas son seis matrices de atención independientes sobre seis subespacios de 32 dimensiones, calculadas en paralelo y luego **concatenadas**:

$$\text{MHA}(x)_i = W_O \left[\sum_j \alpha_{ij}^{(1)} v_j^{(1)} \parallel \cdots \parallel \sum_j \alpha_{ij}^{(6)} v_j^{(6)} \right] \quad (27)$$

El W_O (output_projection, 192×192) **no es decorativo**: hasta el **reshape** las cabezas viven en bloques disjuntos de coordenadas y no se han hablado nunca. Es la única capa que combina lo que trajo la cabeza 0 con lo que trajo la cabeza 4.

El trade-off es exacto porque $h \cdot d_{\text{head}} = d_{\text{model}}$ es constante: más cabezas = más patrones de enrutamiento simultáneos, pero cada uno con menos resolución para comparar. Con ≤ 17 posiciones, 6 cabezas de 32 van holgadas.

Paso 5 — de aquí sale un $\Delta m / \Delta t$. Es el argumento de diseño completo y tiene tres eslabones:

1. **En los logits, selección por diferencia temporal.** Como x_i contiene $\tau_i \mathbf{u}$ de forma aditiva (Ec. (22)) y Q, K son mapas lineales de $\text{LN}(x)$, el logit contiene un término **bilineal** en los dos tiempos:

$$\ell_{ij} \supset a \tau_i \tau_j + b \tau_i + c \tau_j + (\text{contenido}) \quad (28)$$

un polinomio de segundo grado en (τ_i, τ_j) , módulo la no linealidad de la **LayerNorm**. Con eso una cabeza puede implementar “atiende al token de la misma banda unos 5 d más tarde”.

2. **En los values, transporte.** El value de j lleva a la vez su magnitud m_j y su tiempo τ_j , porque ambos entraron linealmente en x_j .
3. **En el residual, la aritmética.** Tras $x_i + \sum_j \alpha_{ij} v_j$ el vector contiene (m_i, τ_i) propios y (m_j, τ_j) importados, en subespacios distintos de las 192 dimensiones. Una capa lineal forma $m_i - m_j$ y $\tau_i - \tau_j$; el GELU del FFN aproxima el cociente.

Corrección importante y muy preguntable: la atención *no* resta. Como $\alpha_{ij} \geq 0$ y $\sum_j \alpha_{ij} = 1$, la salida de la atención es una **combinación convexa** de los values: por sí sola no puede calcular ninguna diferencia. La formulación correcta es

$$\text{atención} = \text{transporte}, \quad \text{residual} + \text{FFN} = \text{aritmética}.$$

Lo que hace $x + \text{MHA}(\cdot)$ es dejar en un mismo vector la información propia del token y la importada del otro, en direcciones distintas; la resta la hace después una capa lineal.

Y por qué esto exige tiempo continuo: si Δt entrara como *positional encoding* por índice de visita, el eslabón 1 codificaría “posición 2 vs. posición 3” en lugar de “5.0 d vs. 4.7 d”, y el eslabón 3 no tendría denominador que dividir.

9.4. Bloque encoder (pre-norm)

$$x \leftarrow x + \text{Dropout}(\text{MHA}(\text{LayerNorm}(x))) \quad (29)$$

$$x \leftarrow x + \text{FFN}(\text{LayerNorm}(x)), \quad \text{FFN}(u) = W_2 \text{Dropout}(\text{GELU}(W_1 u)) \quad (30)$$

La atención es la de la sección 9.3: escalada, **bidireccional** — sin máscara causal, no hay futuro que ocultar porque la ventana entera se ve de una vez — y con la máscara de padding sobre las keys.

Qué compra el pre-norm. La **LayerNorm** va *dentro* de la rama, antes de la atención, y no después del residual. Así el camino residual $x \rightarrow x \rightarrow x$ queda **limpio de normalizaciones** de punta a punta y el gradiente llega intacto a la primera capa. Consecuencia práctica que conviene saber decir: **el warmup de 5 épocas es del scheduler, no un parche de estabilidad**; con post-norm y 6 capas sí haría falta por estabilidad numérica.

Dónde está el dropout y dónde no. Hay dropout (i) sobre la secuencia de entrada completa, (ii) sobre la salida de la atención antes del residual y (iii) dentro del FFN, entre el GELU y la segunda lineal. **No** hay dropout sobre los *pesos* de atención ni sobre la salida del FFN, y `nn.TransformerEncoderLayer` de PyTorch sí lleva los dos: otra diferencia que muerde si algún día se sustituye el bloque.

No hay LayerNorm final. `TransformerEncoder.forward` apila las 6 capas y devuelve el stream residual *sin normalizar* — habitual en pre-norm, pero suele cerrarse con una norma final. Aquí ese papel lo hace `classification_norm`, y **solo en la posición 0**:

`classification_head(LayerNorm(encoded[:,0]))` $\rightarrow$ logits de 2 clases

Es correcto porque solo se lee [CLS], pero cualquier lectura futura de otra posición (pooling sobre tokens, sondeo por banda) saldría sin normalizar.

9.5. Hiperparámetros entrenados

parámetro	<i>default</i> de <code>model.py</code>	entrenamiento actual
<code>d_model</code>	64	192
<code>num_heads</code>	4	6
d_{head} (derivado, $= d_{\text{model}}/h$)	16	32
<code>num_layers</code>	2	6
<code>d_feedforward</code>	128	768 ($4 \times d_{\text{model}}$)
<code>dropout</code>	0.1	0.1
<code>num_classes</code>	2	2 (other, KN)
parámetros totales	—	2 674 225

En qué se gastan esos 2.67 M. Recalculado desde el código, cuadra al dígito:

componente	parámetros
<code>qkv_projection</code> ($192 \rightarrow 576$)	111 168
<code>output_projection</code> ($192 \rightarrow 192$)	37 056
<code>attention_norm</code> + <code>feedforward_norm</code>	768
FFN ($192 \rightarrow 768 \rightarrow 192$)	295 872
por bloque	444 864
× 6 capas	2 669 184
<code>TokenEmbedding</code> (banda 36, tipo 9, proyección 2 688, norm 384, Time2Vec 386)	3 503
<code>GlobalTokens</code> (CLS 192, no_redshift 192, proyección de z 384)	768
<code>classification_norm</code> + <code>classification_head</code>	770
total	2 674 225

Dato con el que defender la arquitectura: **el 66 % de cada bloque es el FFN, no la atención.** Con ≤ 17 posiciones la matriz de atención tiene 289 entradas por cabeza y es computacionalmente irrelevante; el modelo gasta su capacidad en *transformar* tokens, no en enrutarlos.

9.6. Receta de entrenamiento

- Optimizador **AdamW**; scheduler **coseno con warmup** lineal (5 épocas); lr 10^{-3} , weight decay 10^{-4} , batch 512.
- Hasta 150 épocas con **EarlyStopping** (patience 25) monitorizando `val_acc_noz`; la corrida registrada llegó a la **época 122** (~ 647 k pasos).
- `val_acc_noz` no es un número suelto: es la **media de la macro-accuracy sobre los tres regímenes sin z** (1, 2 y 3 épocas). O sea, el checkpoint se elige por rendimiento equilibrado a lo largo del presupuesto de épocas, no por la ventana completa.
- **Pesos de clase** inversos a la frecuencia con `mode='sqrt'` (suaviza el desbalance en vez de invertirlo del todo).
- **Selección de modelo por `val_acc_noz`**, no por `val_acc_z`.

- **Model soup:** se promedian los pesos de los 5 mejores checkpoints en vez de fiarse de una sola época (`checkpoints/kilonova_transformer-soup.ckpt`).

¿Por qué seleccionar por `val_acc_noz`? Porque KN y contaminantes son casi *disjuntos en redshift*: las KN de LANL se concentran brutalmente a z bajo (mediana 0.09, p90 0.37, p99 0.77) mientras los contaminantes llegan a $z \sim 3$. Con z visible, la exactitud sale artificialmente casi perfecta: el modelo puede usar el atajo “ z bajo $\Rightarrow$ KN”, que solo existe porque el batch está balanceado — en la realidad, incluso a $z < 0.5$, los contaminantes superan a las KN $\sim 100:1$. La dirección contraria (“ z alto $\Rightarrow$ no es KN”) sí es física y debe aprenderse.

Pero no digas “las KN solo se detectan bajo $z = 0.5$ ”, porque no es cierto y es fácil de refutar: el 4.7 % de las KN detectadas en deep está por encima, y a $z = 1.0$ todavía se detectan 809 de cada 10 000. Lo correcto es “el 95 % se detecta bajo $z = 0.5$, con cola hasta $z = 1$ ”. (El comentario de `train_lightning.py` que dice “KN capped at $z = 0.4$ ” está mal.)

9.7. Split anti-fuga

90/5/5, y la regla es una sola: **nunca se reparten objetos, se reparten grupos**, y el grupo es *global a los dos tiers*. Lo que cambia entre clases es qué se considera un grupo:

- **KN** $\rightarrow$ grupo = `simulation_id` (el primer campo del `object_id`). Si la misma simulación física apareciera en train y test, con otro ángulo, otro z u otro ruido, sería fuga. Son 900 grupos $\rightarrow$ 810/45/45 modelos.
- **Contaminantes** $\rightarrow$ grupo = `object_id (snana_{healpix}_{id})`, *sin* prefijo de tier, estratificado por clase original.

Como las fracciones se aplican al número de *grupos* y no todos los grupos pesan lo mismo (un contaminante visto en los dos tiers vale 2 objetos, uno visto solo en deep vale 1), los tamaños en objetos se desvían un poco del 90/5/5 exacto: **2 693 717 / 151 961 / 153 702** (test: 79 926 other + 73 776 KN).

La fuga que hubo, y hay que saber contestarla (arreglada el 2 de agosto de 2026; el modelo de la sección 10 es *anterior* al arreglo). El fichero *deep* es superset del *wide*: **717 863** transientes de OpenUniverse están en los dos tiers con el *mismo* `object_id`. La versión vieja del split le ponía prefijo de tier a la clave de grupo (`deep_{id}` vs `wide_{id}`), así que las dos vistas del *mismo* transiente caían en splits **independientes**. Con 90/5/5 eso son

$$717\,863 \times 2 \times 0.9 \times 0.05 \approx 64\,600$$

objetos de test (~ 81 % de los 79 885 contaminantes de test de entonces) cuyo gemelo del otro tier estaba en train: misma curva SNANA, mismo z , mismo pico; solo cambian bandas, exposición, ruido y ventana. Era exactamente lo contrario de lo que ya se hacía con las KN, agrupadas por `simulation_id` a través de los tiers a propósito. **El arreglo es quitar el prefijo**; medido después, los objetos de test con gemelo en train pasan de $\sim 64\,600$ a 0.

Por qué el prefijo estaba ahí, que es la pregunta de verdad: para que los ids de deep y wide “nunca colisionaran”. Pero colisionar era justo lo que se necesitaba — el id *ya* es único a nivel de transiente porque lleva el healpix dentro. La lección general: una clave de grupo namespaced por vista destruye el agrupamiento en cuanto la misma entidad tiene varias vistas.

10. Resultados

Aviso de procedencia. Todos los números de esta sección son del checkpoint *soup* entrenado sobre el split **anterior** al arreglo de la fuga entre tiers (sección 9.7), en el que ~ 81 % de los contaminantes de test tenía su gemelo del otro tier en train. **Son por tanto un techo optimista sobre la clase other** y hay que volver a medirlos tras el re-entrenamiento. El lado KN no está afectado: las KN siempre se agruparon por `simulation_id` a través de los tiers.

Sobre el checkpoint *soup*. Split (v1): 2 693 804 train / 151 915 validación / 153 661 test. Balance del test: 79 885 *other*, 73 776 KN.

10.1. Matriz de degradación (sin redshift)

épocas	clase	precision	recall	F1	accuracy
1	other	0.8711	0.9356	0.9022	0.8945
1	KN	0.9242	0.8501	0.8856	
2	other	0.9492	0.9780	0.9634	0.9614
2	KN	0.9754	0.9433	0.9591	
3	other	0.9841	0.9913	0.9877	0.9872
3	KN	0.9905	0.9827	0.9866	

10.2. Recall de KN en el punto de operación *recall-first*

Umbral $P(\text{KN}) \geq 0.2$ en vez del 0.5 de la matriz anterior (THRESHOLD_HIGH_RECALL en `run_evaluation_test_only`)

épocas	sin redshift		con redshift	
	recall KN	contaminantes como KN	recall KN	contaminantes como KN
1	0.9239	16 859	0.9821	4 927
2	0.9659	6 494	0.9955	1 140
3	0.9916	2 431	0.9987	419

Lectura: la información llega rápido — ya con *una sola época* el recall sin z supera el 92 %, y la tercera época divide los falsos positivos por ~ 7 . El redshift ayuda, pero recuérdese la advertencia del recuadro anterior: parte de esa ayuda es el atajo del prior, no física.

11. Apéndice: preguntas típicas

1. **¿Por qué el denominador del ruido de lectura es $n(n+1)$?** Porque se ajusta la pendiente de la rampa sobre n lecturas no destructivas; con $(n+1)$ el resultado es $9\times$ mayor (55.46 vs 5.942 e^-) y no reproduce la Tabla 3. Ese 5.942 es a los 900 s de la Tabla 3; a los 1636 s reales de F184 deep son 5.540 e^- .
2. **¿Por qué σ_{read} baja al aumentar la exposición?** *Up-the-ramp*: más tiempo $\Rightarrow$ más lecturas $\Rightarrow$ mejor estimación de la pendiente. Satura en el piso $\sqrt{25} = 5 e^-$.
3. **¿Cuál es el tiempo mínimo y máximo de la grilla LANL?** 0.125 d (3 h) a 64 d en reposo, 73 nodos logarítmicos de razón 1.0905. Pero solo 100 de las 900 simulaciones llegan a 64 d: por simulación son 63–73 nodos y el último tiempo cae entre 26.9 y 64 d.
4. **¿Qué implica el corte de 0.125 d?** Que no hay ninguna predicción para las primeras 3 horas post-merger, y en el observador ese corte se estira a $0.125(1+z)$ d. Afecta sobre todo a las bandas azules, que es donde la KN brilla más al principio.
5. **¿Qué distingue a `run_type` de `wind`?** `run_type` (TP/TS) es la *morfología* del viento — “peanut” o esférica — con eyecta dinámica siempre toroidal; `wind` (`wind1/wind2`) es la *composición* del viento, $Y_e = 0.27$ o 0.37. La dinámica tiene $Y_e = 0.04$ fijo y no es eje de la grilla.
6. **¿Por qué el ángulo de visión cambia tanto la curva?** Porque la eyecta dinámica es toroidal, rica en lantánidos y opaca: por el ecuador tapa el viento azul, por el polo no. Dos ángulos de la misma simulación pueden diferir en varias magnitudes.

7. **¿Qué es el *dimming* cosmológico y cuántos factores $(1+z)$ tiene?** Dilución geométrica $1/(4\pi d^2)$ más dos factores $(1+z)$ — energía por fotón y tasa de llegada — ya absorbidos en $d_L = (1+z)d_C$. Para fuentes puntuales son *dos*, no cuatro; los cuatro $((1+z)^{-4})$ son del brillo superficial de fuentes extensas.
8. **¿De dónde sale el factor 54 y cuánto vale en magnitudes?** $4\pi/\Delta\Omega_{\text{bin}}$ con bins uniformes en $\cos\theta$; $-2.5\log_{10} 54 = 4.331$ mag. Sin él todas las KN salían 4.33 mag demasiado débiles.
9. **¿Por qué el término de fuente lleva α y el de fondo α^2 ?** Ambos llevan α^2 ; en el de fuente un factor se cancela porque $\text{Var}_e = F_e = F_{\text{fcal}}/\alpha$ (Poisson: varianza = media, lineal en las cuentas). El fondo ya es una varianza, así que α^2 queda entero. No hay factor de Fano.
10. **¿Por qué NEA tabulada y no `roman.getPSF`?** La PSF analítica del Ciclo 9 de galsim sobrestima el NEA un 15–28 % en R/Z/Y.
11. **Diferencia entre un token u y uno n.** u: se miró y no se detectó $\Rightarrow$ la fuente es *más débil* que el límite 5σ de esa visita (información positiva). n: no se miró $\Rightarrow$ ninguna información sobre el brillo, pero sí sobre la cadencia.
12. **¿Por qué el límite 5σ es por visita y no una profundidad fija de survey?** Porque la profundidad realmente alcanzada es el contenido informativo de la no-detección; y usar un valor fijo para las KN inyectadas convertiría el límite en un delator del pipeline.
13. **¿Por qué no hay token de modo deep/wide?** Lo que llevaría (la profundidad) ya está en σ_m y en m_{lim} de forma continua; un token de modo grabaría la rejilla del simulador en el encoder y ampliaría la brecha de dominio con Roman real.
14. **¿Por qué el tiempo va dentro del token y no como positional encoding?** Los tokens son un *conjunto*, no una secuencia. Solo con una representación continua del tiempo la atención puede restar dos tokens y calcular $\Delta m/\Delta t$.
15. **¿Por qué Time2Vec con 0 frecuencias?** Tres épocas sobre ~ 15 d no resuelven ninguna frecuencia, y los transientes no son pulsadores. Solo el término lineal tiene contenido físico. Con $K = 0$ el encoding es un mapa afín de *rango 1*: el tiempo ocupa una sola dirección de las 192.
16. **¿Qué son *query*, *key* y *value*, y por qué $\text{key} \neq \text{value}$?** Query = lo que una posición busca, key = el índice con que otra se anuncia, value = lo que entrega si la eligen. Key y value son distintos porque el *criterio de selección* y el *contenido transportado* son independientes: un token se anuncia por (banda, Δt) y entrega (m , σ_m). El [CLS] no tiene contenido propio — es un parámetro aprendido — y su único papel es emitir la query que recoge la evidencia.
17. **¿La atención calcula el $\Delta m/\Delta t$?** No por sí sola: sus pesos son positivos y suman 1, así que solo produce combinaciones *convexas* de los valores — transporta, no resta. La resta la permite el residual, que deja el valor propio y el importado en el mismo vector, y la hace la capa lineal siguiente.
18. **¿Por qué se divide por $\sqrt{d_{\text{head}}}$ y no por d_{head} ?** Porque $q \cdot k$ es una suma de $d_{\text{head}} = 32$ productos y su desviación crece como $\sqrt{32}$. Sin el factor, el softmax nace saturado (*one-hot*) y no hay gradiente para aprender a mezclar. Es normalización de varianza.
19. **¿Qué enmascara exactamente `attention_mask`?** Solo las *keys* (columnas): su forma $(B, 1, 1, L)$ hace broadcast sobre las filas. Las posiciones de padding siguen generando salida, pero nadie la lee. Como CLS y Z son siempre válidas, ninguna fila queda entera enmascarada y el -9×10^{15} nunca produce NaN.

20. **¿Por qué se selecciona por `val_acc_noz`?** Ver el recuadro de la sección del transformer: KN y contaminantes son casi disjuntos en z , así que `val_acc_z` es un artefacto del batch balanceado.
21. **¿Por qué el split separa KN por `simulation_id` y contaminantes por `object_id`?** Porque el grupo tiene que ser la *entidad física repetida* en cada rama. En las KN la misma simulación LANL genera muchas realizaciones (ángulo, offset, z , paridad, ruido). En los contaminantes cada transiente SNANA es único, pero se observa dos veces (deep y wide). En los dos casos el grupo es global a los tiers.
22. **¿Qué hace la paridad de cadencia y por qué se sortea?** Fija si la primera visita es par o impar, es decir qué par de filtros se observa. Sin sortearla, la clase KN quedaba marcada (92 % par) y la sola máscara de bandas clasificaba al 80.7 %.
23. **¿Por qué el ruido se aplica en flujo y no en magnitud?** Porque el ruido es gaussiano en flujo; en magnitud sería sesgado y no está definido para flujos negativos (que ocurren en la cola de no-detecciones).
24. **¿Por qué se guardan 4 épocas si el modelo ve 3?** La cuarta es el *buffer* de la augmentación de desplazamiento, que simula transientes descubiertos ya con ~ 5 d de edad.
25. **¿Cómo puede el modelo clasificar con 1 época si se entrena con 3?** Porque no se entrena solo con 3: el dataloader trunca cada ejemplo a 1, 2 o 3 épocas al azar con probabilidad uniforme. Es la augmentación que sostiene toda la matriz de degradación.
26. **¿Cuántos tokens tiene una secuencia como máximo?** 15 (3 épocas $\times$ 5 bandas del tier, no 6 — el vocabulario de 6 es la unión de deep y wide), más [CLS] y [Z]: 17 posiciones.
27. **¿Qué extinción llevan tus dos clases?** Los contaminantes, la del host (dentro del modelo SED de SNANA); las KN, ninguna. Ninguna de las dos lleva extinción de la Vía Láctea: el catálogo trae `mw_extinction_applied` en False. La cadena F99 host + MW de `extinction.py` pertenece al CLI `kn-extinguish`, que no genera los datos de entrenamiento.
28. **Tu censo dice 1.6 millones de contaminantes. ¿Son 1.6 millones de transientes?** No: son instancias por tier. Los transientes distintos son 879 832; el fichero deep es superset del wide y un mismo objeto observado en los dos cuenta dos veces, porque el dataloader los trata como objetos separados.
29. **¿Qué le pasa a tu split con esos objetos que están en los dos tiers?** Ahora nada: el grupo es el `object_id` desnudo, global a los dos tiers, igual que las KN se agrupan por `simulation_id` a través de los tiers. Hasta el 2 de agosto de 2026 la clave llevaba prefijo de tier y ~ 81 % de los contaminantes de test tenían su gemelo en train; los resultados de la sección 10 son de antes del arreglo.
30. **OpenUniverse ya trae kilonovas. ¿Por qué usar la grilla LANL?** Porque son ~ 2 por healpix, con modelos NON1ASED.KN-K17 y `peak_mag_F` ≈ 30 : nunca alcanzan $S/N \geq 5$, así que no dan estadística ni contaminan la clase *other*.
31. **¿Por qué las magnitudes se normalizan globalmente y no por objeto?** Porque el brillo aparente es señal física (junto con z da la luminosidad); normalizar por objeto la destruiría.
32. **¿Qué aporta OpenUniverse que Hourglass no?** El conjunto completo de 5–6 bandas Roman sin partición wide/deep incompleta, y una estadística de contaminantes de 1.6×10^6 objetos. Hourglass detectó ~ 3 KN en total.

Fuentes dentro del repositorio

`docs/plan.md` (plan científico) · `docs/hourglass_noise_recipe.md` (derivación de la receta y notas del paper) · `docs/kn_detectability_vs_chase.md` (validación externa) · `docs/generate_datasets.md` (runbook) · `src/kilonova/photometry/roman_noise.py` (constantes y primitivas) · `src/kilonova/simulation/early_window` (ventanas, inyección KN) · `src/kilonova/simulation/extinction.py` (extinción y fotometría sintética) · `training/docs/*.md` (tokens, arquitectura, parámetros KN, clase *other*) · `training/model.py`, `training/train_lightning.py`, `training/evaluation.ipynb`.

Externas (datos del observatorio): <https://science.nasa.gov/mission/roman-space-telescope/> · https://roman.gsfc.nasa.gov/science/WFI_technical.html (specs de la WFI) · <https://www.stsci.edu/roman/observatory> · las constantes de `galsim.roman`, que es la implementación que usa el pipeline.